The LP relaxation of the precedence-constrained knapsack problem

Computational report

Valerio Dose Fabio Furini Marco Locatelli

5 September 2026

Abstract

This report documents the software implementation and the experimental study accompanying our work on the LP relaxation of the precedence-constrained knapsack problem. It states what was implemented, how each component was verified, and what the experiments show. The central experimental question is operational: *how should this relaxation be solved in practice?* The answer is regime-dependent, and the regime is set by whether one capacity or the whole value function is needed, and by the density of the precedence graph — not by the number of items. A secondary finding concerns accuracy rather than speed: on one open-pit instance every LP solver we tried returns a value that is wrong in the eighth significant digit, where the combinatorial method is exact. The software, the raw measurements and a browsable edition of this report are at `https://github.com/fabiofurini/macroitems`.

Contents

1 What was implemented

The library `macroitems` (Python, MIT) implements the objects of the theory and the algorithms that produce them. Everything reduces to maximum closure, which is one minimum cut on a Picard network.

1.1 The core

Function	What it returns	Cost
`max_closure`	a maximum-value closure, with the inclusion-wise maximal or minimal tie convention, and optional forcing of items in or out	one minimum cut
`canonical_path`	the canonical macroitem sequence $\mathcal{I}_1, \ldots, \mathcal{I}_k$, its ratios, and hence $z(c)$ for *every* capacity at once; by geometric bisection or by repeated maximum-ratio extraction	$O(k)$ maximum flows on shrinking graphs
`solve_capacity`	the LP at one capacity, by a Newton search on the weight price	a handful of maximum flows
`solution_from_path`	any capacity read off a precomputed path	$O(\log k)$, no flow
`canonical_dual`	the canonical dual certificate: the capacity price and three region-wise flows	three maximum flows
`best_reduced_costs`	reduced costs optimised over the *whole* dual optimal face, and the items a given incumbent lets one fix	one minimum cut per item
`face_dimensions`	dimensions of the primal and dual optimal faces	$\lvert\mathcal{H}\rvert$ minimum cuts
`first_macroitem_lawler`	Lawler's binary search on the ratio, kept for comparison	$O(\log)$ maximum flows

1.2 Infrastructure

Three interchangeable maximum-flow backends.
OR-Tools (int64, exact and the default), igraph (floating point) and SciPy (int32). They must agree on integer data, which the test suite checks. SciPy *refuses* capacities above $2^{31} - 1$ rather than truncating them, for the reason in Section 11.

Exact arithmetic wherever the data allow it.
Instances with decimal data are scaled to integers by reading the number of decimals off each value's shortest decimal representation and scaling in decimal arithmetic. Closures, macroitems and ratios are invariant under that scaling, so the answer does not change, but the parametric values become integers and the whole computation is exact: the breakpoints are recovered as rationals and no tolerance decides which macroitem an item belongs to. Six of the ten MineLib instances and all 23 benchmark instances enter this regime.

Five LP baselines behind one interface.
HiGHS (the reproducible open-source reference), SciPy, Gurobi, CPLEX, and a CPLEX backend that drives a licensed Interactive Optimizer over a pipe — necessary because the CPLEX distribution on PyPI is the Community Edition, which refuses models above 1000 rows, that is, every instance considered here. Each builds its model once and re-solves at further capacities by changing only the capacity right-hand side, so simplex bases carry over; build time and solve time are reported separately. **No third-party solver code is bundled**: the solvers are linked, and installed by the user under their own licences.

Instance readers.
For MineLib and for the precedence-constrained knapsack benchmark, in both of the latter's formats. The MineLib tonnage is determined and *verified* automatically, see Section 3.

A command-line interface and experiment scripts
for the structural characterisation, the method comparison, the revenue-factor study, and the generation of every table and figure in this report from the raw CSV files.

1.3 What was implemented and then disabled

The reduction of the canonical path to a single source–sink-monotone parametric minimum cut is implemented and tested, and then deliberately disabled, because the public implementation it would call is incorrect. See Section 9.

2 What was tested, and how

The suite is 773 tests and runs in about seventy seconds. Its design principle is that every check has an *independent* reference: exact rational arithmetic, a second backend, a second algorithm, or the LP itself. That matters here more than usual, because in this subject a wrong implementation returns something that looks like an answer — nested chains of closures, decreasing ratios, plausible reduced costs.

2.1 Against an exact brute-force reference

Small instances are compared against a reference written in exact rational arithmetic that enumerates *every* closure.

What is checked	Covera
$u(\lambda)$ and both lattice extremes at every breakpoint	$\sim 3\,800$ $(\lambda, \text{instance})$ pairs, 216 instanc
the canonical sequence, from its definition	216 instances, both algorith
$z(c)$ over a capacity grid	136 instances $\times$ a half-integer gr
persistency against the *full* optimal face	1 700+ capacities, of which 250+ with non-unique optir
the dual certificate	80 instanc

The degenerate cases are deliberate: all profits negative, all positive, zero profits, a single item, no arcs, stars, chains, and instances built so that two disjoint continuations have equal ratio. The tie convention is where this subject hides its errors.

2.2 Invariants, on arbitrary instances

Property-based tests (`hypothesis`, ~ 900 generated instances) assert what must hold for *any* instance: prefixes are closures; the macroitems partition the item set; ratios strictly decrease in exact arithmetic; $z(c)$ is concave and nondecreasing; the returned solution is feasible and attains the reported value.

2.3 Between implementations

The two path algorithms must agree (100 random DAGs across five densities and two seeds, plus grids); the three maximum-flow backends must give identical closures and paths on integer data; the LP baselines must agree with the combinatorial methods and with each other to 10^{-9}; and the integer rescaling must preserve ratios while scaling values.

2.4 Against the other literatures

This is the part that tests the *theory*'s claim rather than the code. Each algorithm is reimplemented from its own source, using nothing from the library.

Source	Claim	Coverage
Dantzig (1957)	with no arcs the theory reduces to the greedy ratio rule	67 arc-free instances
Shaw and Cho (1998)	their aggregated subtrees are the macroitems and their bound is $z(c)$	43 tree instances, $\sim$ 1 500 capacities
Sidney (1975); Margot, Queyranne and Wang (2003)	the reduced Sidney decomposition is the canonical sequence, after reversing the arcs and swapping the roles of the data	57 instances, decomposition by brute force over all initial sets

All three hold. They are not vacuous: dropping the arc reversal in the scheduling translation breaks 188 of 190 instances, and using the finest rather than the coarsest tie convention in the Shaw–Cho deletion breaks 8 of the 43 tree cases.

2.5 As a user receives it

The package is built as a wheel, installed into an empty environment, and exercised from the command line, both with all optional backends and with its declared dependencies alone. Continuous integration runs the suite on Python 3.10 to 3.13 and repeats the core-only installation on every push. Two of the defects of Section 11 were found by exactly this.

3 Instances

No instance data is redistributed; the readers take the files as published by their authors.

Collection	Family	Instances	Sizes
PCKP benchmark	telecom (A–K)	11	$n = 972$–$9\,235$, $m/n \approx 1.8$
	mining (L–W)	12	$n = 349$–$11\,757$, $m/n \approx 6$–7
MineLib		10	$n = 1\,060$–$112\,687$, up to $m = 3\,035\,483$

Two points about the files are worth recording, since both cost time to establish.

The MineLib tonnage is not labelled. The block-model files carry between 8 and 19 unlabelled attributes and the format specification calls them "optional user-specified fields". The reader determines the tonnage and verifies it: every block must be moved, so the tonnage is positive on *all* blocks; an operational resource of the constrained-pit file whose coefficients cover every block is therefore the mining resource, and is used directly (8 of the 10 instances). A resource covering only part of the blocks is a processing resource, and then the tonnage is the block attribute that is positive everywhere *and* agrees with those partial coefficients wherever they exist — the agreement identifies the column, since a processing resource charges an ore block exactly its tonnage. This settles `kd` (agreement on all 5 931 ore blocks) and `mclaughlin_limit` (on all 31 931). Each instance records how it was decided.

The benchmark's two formats disagree on item numbering. On 13 of the 23 instances the LP-format and tabular files number the items differently. They are the same instances — the multisets of profits and weights agree exactly, and so do the arc counts — but item indices are not comparable across formats. Separately, the family labels in the `runList` files distributed with some copies of the accompanying code are the opposite of the original distribution's; the data settle it, since the mining instances are the ones with negative-profit waste.

4 Setup and protocol

Intel Core i5-14500T, 14 GB, Ubuntu 26.04, Python 3.14.4; OR-Tools 9.15, igraph 1.0, HiGHS 1.15.1, Gurobi 13.0.3, CPLEX 22.1 (Interactive Optimizer, licensed).

Every run is single-threaded. Each method runs in its own process — required for correctness, since `highspy` and `ortools` export incompatible copies of the same C++ symbols and only the one imported first loads, and good practice for timing in any case. Build time is separated from solve time, and the LP baselines build once and then change only the capacity right-hand side, so their simplex bases carry over; a baseline that rebuilt its model at every capacity would be measuring the wrong thing. Capacities are evenly spaced inside $(0, w(\mathcal{M}_q))$, the range on which the capacity constraint binds. Every method's values are compared against the canonical path at every capacity, and any relative difference above 10^{-9} is reported rather than hidden.

5 The structure of real instances

The fraction $1 - |\mathcal{H}|/n$ of items whose value is the same in every optimal solution — the persistency — is what the relaxation settles on its own. It varies enormously.

	Instance	Persistency	Split macroitem
lowest	O_1711_11661	0.150	1 455 of 1 711 items
	newman1	0.166	884 of 1 060 blocks
	U_6494_48626	0.262	4 790 of 6 494
highest	p4hd	0.992	329 of 40 947
	sm2, zuck_large	0.9999	10 of ∼100 000

On `sm2` and `zuck_large` the relaxation fixes all but ten blocks out of a hundred thousand; on `newman1` it fixes almost nothing. This is the *gap problem* of the mining literature, measured rather than described, and the theory says exactly what separates the two groups: the weight of the split macroitem relative to the capacity.

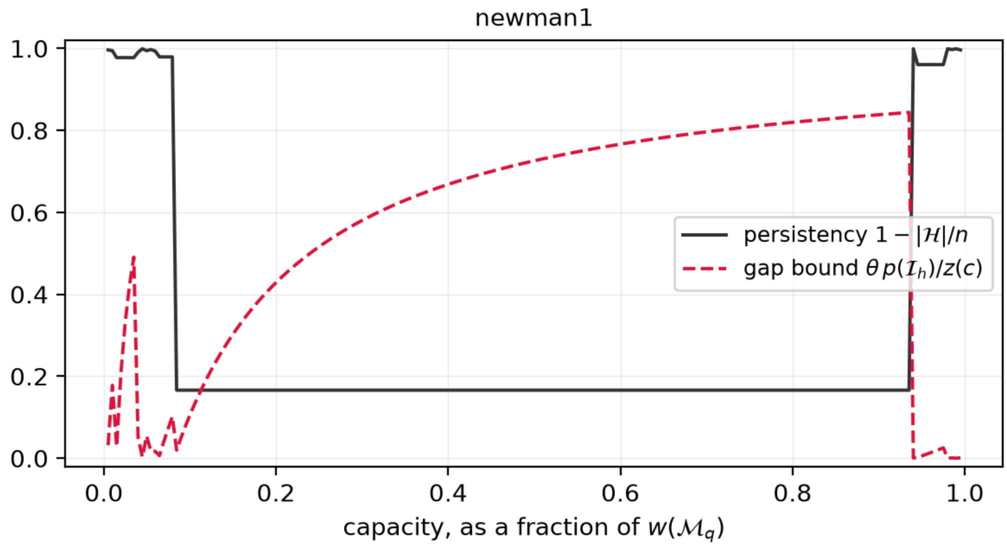

Figure 1: Persistency and the integrality-gap bound across the capacity range of `newman1`. Persistency is near 1 only below 8% of $w(\mathcal{M}_q)$; over the entire practical range it collapses to 0.166 while the gap bound climbs to 0.84. One macroitem dominates the instance.

The published capacities sit in the hard regime. On *all* 23 benchmark instances the distributed capacity is between 0.5% and 25% of $w(\mathcal{M}_q)$, so the split macroitem is the *first* one. The optimal solution is then a multiple of a single macroitem's indicator, the integrality-gap bound is vacuous, and the true gap against the known integer optima is large: median 26.6%, up to 85.6%. The standard benchmark of this problem exercises precisely the regime in which the natural relaxation is weakest, which is consistent with that literature's focus on cutting planes.

6 How best to solve the relaxation

6.1 The whole value function

The canonical path returns *every* capacity at once; a further capacity then costs a binary search over the breakpoints. An LP solver answers one capacity per solve. Over twenty capacities on the benchmark, against the fastest of the three simplex codes:

Group	Instances	Median ratio	Range
sparse, $m < 10^4$	12	1.00×	0.47–2.44
dense, $m \geq 2 \cdot 10^4$	7	**11.2**×	5.0–29.0

The largest benchmark instance, $n = 11\,757$ and $m = 83\,218$, takes 0.42 seconds for the whole of $z(\cdot)$ against 12.2 seconds for one grid of solves, a factor of 29.

Density, not size, is the discriminant. The sparse group contains instances with 9 235 items on which the methods tie, while an instance with 3 243 items but 22 306 arcs already gives a factor of 8. What separates them is the precedence structure a simplex basis has to carry.

6.2 The open-pit instances

On MineLib the same effect is much larger. Table 2 gives the whole comparison, at ten capacities and a budget of 150 seconds per method.

Table 2: MineLib: total time for ten capacities. *best solver* is the fastest of the three dual simplex codes; TO means that none of them finished a single capacity within 150 s.

Instance	n	m	path (s)	Newton (s)	best solver (s)	ratio
newman1	1 060	3 922	0.04	0.15	0.1	1.4×
zuck_small	9 400	145 640	0.45	2.17	19.6	43.8×
kd	14 153	219 778	1.16	2.19	42.0	36.1×
zuck_medium	29 277	1 271 207	3.16	18.88	299.5	**94.9**×
p4hd	40 947	738 609	11.36	7.01	TO	—
marvin	53 271	650 631	3.34	5.24	25.7	7.7×
w23	74 260	764 786	12.41	14.88	TO	—
zuck_large	96 821	1 053 105	25.00	30.91	TO	—
sm2	99 014	96 642	2.38	1.45	1.2	0.5×
mclaughlin_limit	112 687	3 035 483	46.50	39.91	TO	—

Three readings.

Where the solvers finish, the margin is an order of magnitude or more: a median of 21.9× and a maximum of 94.9×, on `zuck_medium`, where the path returns the whole value function in 3.2 seconds against 299 seconds for a warm-started dual simplex.

On four of the ten instances no simplex code finished a single capacity within 150 seconds, while the path computed *every* capacity in 11 to 47 seconds. On `mclaughlin_limit` — 112 687 blocks and 3 035 483 precedences — that is 46.5 seconds for the entire value function against no answer at all.

The one instance the solvers win is the sparsest, and that is the point. `sm2` has 99 014 items but only 96 642 arcs, a density below 1, and there a dual simplex is twice as fast as the path. It is the largest instance in the collection by item count and the only one on which the combinatorial method loses — which is the density thesis of Section 6 stated by a single instance.

6.3 One capacity

The Newton search on the weight price is a median 1.5× faster than the fastest simplex code, with a wide range (0.38–26.9×): on small sparse instances a warm-started dual simplex is at least as good, and there is no reason to prefer the combinatorial route there.

6.4 Interior point

Barrier and IPM cannot start from a previous basis, so their marginal cost per capacity equals their full cost; over a grid they were 10–30 times worse than dual simplex in every run, and they are the first to hit a time limit on the large instances. Worth stating, because barrier is a common default on large sparse LPs and here it is the wrong default.

6.5 Recommendation

Situation	**Use**	**Why**
one capacity, sparse graph	either	they tie
one capacity, dense graph	Newton on the weight price	a handful of cuts on shrinking graphs
more than a couple of capacities	the canonical path	further capacities are free
an LP solver is required	dual simplex, never barrier	only simplex warm starts
the value must be exact	the combinatorial method	Section 7

7 Accuracy

Across 161 method-runs on the 23 benchmark instances, twenty capacities each, the largest relative difference between any two methods is $4.8 \cdot 10^{-11}$.

On MineLib `kd` it is not. At one capacity out of twenty, all five LP solver configurations return 220 239 941.533204 where the exact value, computed in rational arithmetic from the integer-scaled instance, is

$$\frac{1\,238\,890\,176\,810\,816\,816\,310\,940\,159}{5\,625\,184\,000\,000} = 220\,239\,938.251054,$$

a relative error of $1.5 \cdot 10^{-8}$ where at the other nineteen capacities the same solvers agree to 10^{-15}. It is not an artefact of the integer rescaling: Gurobi returns the same wrong value on the original decimal data. The instance has profits of order $4 \cdot 10^5$ against weights of order 10^4, and at that capacity it is ill-conditioned enough for a double-precision solver's default tolerances.

An error of 10^{-8} is harmless when the LP value is a report. It is not harmless when it is a *bound*: inside an enumerative algorithm a bound too large prunes nothing it should, and a bound too small prunes an optimal subtree. Nor is it harmless when the quantity of interest is a difference of two LP values, as an integrality gap or a reduced cost is. This is the argument for the combinatorial route that a table of running times does not show.

8 Dual certificates and reduced costs

The canonical dual certificate is three maximum flows, one per region, and costs about as much as the primal solution. On every instance it was feasible and complementary to machine precision — a meaningful check, since the feasibility conditions of the three flow systems are exactly the optimality of the two extremal closures together with the maximum-ratio property of the split macroitem, so a failure would indicate an error in the *primal*.

Reduced costs optimised over the whole dual optimal face need one minimum cut per item: for a null item, a maximum closure on the subgraph induced by the null region with that item forced in; for a full item, the same computation on the full region *with the arcs reversed*, since a co-closed set of a graph is a closed set of its reverse. The two cases minimise different objectives, and confusing them yields bounds that are invalid but entirely plausible (Section 11).

The face-wide values are never smaller than the closed-form canonical ones $w_i|\lambda_r - \lambda_h|$, and on real instances they are larger by a wide margin: across the 23 benchmark instances the median item gains a factor of 44.5, with the per-instance median between 27.9 and 160.6. Since a reduced cost fixes a variable only when it exceeds the gap to an incumbent, that is the difference between a bound that fixes nothing and one that fixes much of the instance.

The two are not comparable in cost. The canonical values follow from the canonical sequence for nothing; the face-wide ones need one minimum cut per item, 0.8 to 19 seconds on those instances, against 1 to 37 milliseconds for the canonical dual certificate itself. They are what one computes at a node worth the effort, not at every node.

Both quantities were validated against the exact loss obtained by re-solving the relaxation with each item forced to its opposite bound; over twelve instances, no violation.

A third measurement is worth recording because it decides how much room the face-wide values have. On *all* 23 benchmark instances the primal optimum is unique — $k_0 = 1$ and $\dim \mathcal{X}^\star = 0$ — so the intermediate optimal closures that would make the primal face positive-dimensional do not occur there at all, and the dual faces are correspondingly large, with $\dim \mathcal{D}^\star$ from 1 482 to 75 812.

9 Parametric implementations

The natural third family of methods is a parametric minimum cut returning all breakpoints in one computation. We intended to include it and cannot.

The public `pseudoflow` package states in its own documentation that it is a simplified variant of parametric HPF, without free runs or warm starts, and "should not be used" for comparison with the full algorithm. More seriously, it *silently returns an incomplete parametric family.* The smallest failing instance has three items, no precedence arcs, profits $(3, 2, 1)$ and unit weights, whose canonical sequence is $\{1\}, \{2\}, \{3\}$ with ratios $3 > 2 > 1$: it reports two intervals instead of three, and the set returned for the interval containing $\lambda = 7/2$ has value $-1/2$, so it is not a minimum cut. Failures begin at $n = 13$ and are common by $n = 200$.

What makes this dangerous is that the output looks correct: the family returned is always nested, complete, and made of genuine closures with decreasing ratios — a *coarsening* of the canonical sequence. Detecting the defect costs one maximum closure per macroitem, which is recomputing the answer by another method. We therefore report no row for it. There is likewise no public implementation of the Gallo–Grigoriadis–Tarjan algorithm that we would trust for benchmarking, so published running times for parametric minimum cut on this problem are difficult to reproduce independently.

One correction to the textbook reduction, found while implementing it: the offset $K \geq \max_i(-p_i)$ usually quoted does not suffice. The parameter range must reach below the smallest ratio, which is negative whenever some p_i/w_i is, and the sink capacity $\lambda w_i + K$ then goes negative, at which point the underlying C library terminates the process.

10 Weight against revenue factor

Nested pits are usually generated in practice by scaling revenue at fixed cost. That family is nested too, but it is a different family, and only the weight parameterisation solves the relaxation of a tonnage-constrained problem. On block models whose revenue and cost are known by construction, over a grid of twenty revenue factors, only 5 of 20 revenue-factor pits coincide with a canonical closure, and the relative symmetric difference to the canonical closure of the nearest tonnage reaches 0.37: more than a third of the blocks differ between the two pits of the same size.

Repeating this on MineLib is blocked by the data rather than the method: the ultimate-pit and constrained-pit files give a single value per block, from which revenue and cost cannot be separated; only the production-scheduling formulation lists a value per destination.

11 Defects found during development

Recorded because the same failure modes are available to anyone reimplementing these classical algorithms, and because *every one of them produced a plausible answer rather than an error.*

1. **Reduced costs, sign inverted on the full region.** The two regions minimise different objectives; getting the second wrong produced reduced costs that were positive, sensibly ordered and of the right magnitude, and invalid. Caught by comparing against the exact loss.
2. **An infeasible point at $c = 0$.** A negative Python slice wrapped around and returned all but the last macroitem — positive weight for zero capacity. Caught by property-based testing at the boundary.
3. **The SciPy maximum-flow backend truncated silently.** It computes in int32 and truncates larger capacities without warning, returning a *different canonical path* on integer data with wide coefficients. Caught by requiring the three backends to agree; such capacities are now refused.
4. **Ratios divided by the integer scale.** A ratio is invariant under rescaling; the cumulative profits and weights are not. Reporting code divided them anyway, which would have published breakpoints wrong by orders of magnitude with every other number in the same table looking right. Caught by drawing a figure and noticing the colour bar.
5. **The package did not install.** A deprecated licence classifier alongside a modern licence expression makes the build back end refuse the project outright. Caught by building a wheel and installing it.
6. **Declared dependencies were not sufficient.** The dual certificate and the face dimensions used floating-point node values that only an optional backend accepts. The remedy was not a new dependency but exactness: on integer data the multiplier is the rational $p(\mathcal{H})/w(\mathcal{H})$, so multiplying the node values by $w(\mathcal{H})$ makes them integers without changing which sets are optimal or tight. Caught by a continuous-integration job on its first execution.

The pattern is worth naming. In this subject a broken implementation returns something that looks like an answer. The only checks that caught anything were those with an independent reference: exact rational arithmetic, another backend, another algorithm, or the LP itself.

12 Reproducing this report

```
git clone https://github.com/fabiofurini/macroitems && cd macroitems
pip install -e ".[experiments]"
pytest -q                                        # 773 tests

python experiments/characterize.py    <instances> --out .../characteristics.csv
python experiments/compare_methods.py <instances> --capacities 20 --out .../compare.csv
python experiments/revenue_factor.py  grid:20x20x8:5:1 --factors 20
python experiments/make_tables.py
python experiments/make_figures.py    <instances>
```

Timings depend on the machine. The *values* do not: any relative difference above 10^{-9} between two methods is flagged in the output, and if it is not the `kd` case of Section 7 it is a defect, in this library or in a solver, and we would like to hear about it.

A Results in full

The tables below are generated from the raw CSV files in `experiments/results/` by `experiments/make_tables` no number is hand-typed. Table 3 is the structural characterisation of every instance, Table 4 the complete method comparison, and Table 5 the recommendation those numbers support.

In Table 3, k is the number of macroitems and q the last one with positive ratio; *largest* is the size of the largest macroitem; $|\mathcal{H}|$, *persist.* and *gap bound* are taken at the instance's own capacity, where it has one.

In Table 4, *first* is the cost of answering one capacity from nothing, *per extra* the marginal cost of each further capacity, and *break-even* the number of capacities beyond which computing the whole canonical path is the cheaper choice. A status of `timeout` means the method did not finish one capacity within the budget, which on the largest instances is itself the result.

Table 3: Structure of the canonical macroitem sequence, and the split macroitem at each instance's own capacity

instance	family	n	m	k	q	largest	|H|	persist.	gap bound
newman1	minelib	1060	3922	47	46	884	884	0.1660	63.46%
zuck_small	minelib	9400	145640	177	176	1484	1484	0.8421	100.00%
kd	minelib	14153	219778	493	483	2215	-	-	-
zuck_medium	minelib	29277	1271207	138	128	12711	12711	0.5658	85.80%
p4hd	minelib	40947	738609	2446	1468	2547	329	0.9920	0.94%
marvin	minelib	53271	650631	270	150	40084	1736	0.9674	100.00%
w23	minelib	74260	764786	2428	1258	24769	24769	0.6665	37.16%
zuck_large	minelib	96821	1053105	3192	3192	16660	10	0.9999	0.08%
sm2	minelib	99014	96642	1667	1538	77736	10	0.9999	0.33%
mclaughlin_limit	minelib	112687	3035483	1884	1790	9409	-	-	-
L_349_2101	mining	349	2101	4	3	170	144	0.5874	100.00%
M_538_3033	mining	538	3033	5	3	262	235	0.5632	100.00%
N_1217_7616	mining	1217	7616	42	29	256	256	0.7896	100.00%
O_1711_11661	mining	1711	11661	3	2	1455	1455	0.1496	100.00%
P_3243_22306	mining	3243	22306	9	5	1393	1355	0.5822	100.00%
Q_3428_19555	mining	3428	19555	187	128	545	545	0.8410	100.00%
R_4281_24452	mining	4281	24452	243	163	712	712	0.8337	100.00%
S_5624_36504	mining	5624	36504	124	61	1097	1097	0.8049	100.00%
T_6271_42080	mining	6271	42080	110	80	1530	1530	0.7560	100.00%
U_6494_48626	mining	6494	48626	18	12	4790	4790	0.2624	100.00%
V_10001_63944	mining	10001	63944	257	147	1862	1862	0.8138	100.00%
W_11757_83218	mining	11757	83218	78	50	3378	3378	0.7127	100.00%
A_972_1661	telecom	972	1661	86	86	271	130	0.8663	100.00%
B_981_1688	telecom	981	1688	77	77	238	53	0.9460	100.00%
C_1336_2382	telecom	1336	2382	88	88	202	202	0.8488	100.00%
D_1790_3130	telecom	1790	3130	94	94	147	103	0.9425	100.00%
E_1790_3130	telecom	1790	3130	94	94	147	103	0.9425	100.00%
F_3091_5715	telecom	3091	5715	494	494	256	51	0.9835	100.00%
G_3091_5715	telecom	3091	5715	106	106	302	302	0.9023	100.00%
H_3091_5715	telecom	3091	5715	515	515	222	46	0.9851	100.00%
I_3091_5715	telecom	3091	5715	160	160	192	192	0.9379	100.00%
J_9235_17082	telecom	9235	17082	601	601	322	271	0.9707	100.00%
K_9235_17082	telecom	9235	17082	613	613	366	96	0.9896	100.00%

Table 4: Cost of the first capacity, of each further capacity, and the number of capacities beyond which computing the whole canonical path is cheaper

instance	n	method	first (s)	per extra (s)	20 caps (s)	break-even
L_349_2101	349	path	0.030	0.0000	0.03	
L_349_2101	349	newton	0.031	0.0030	0.09	1
L_349_2101	349	highs:dual-simplex	0.021	0.0007	0.03	15
L_349_2101	349	highs:ipm	0.025	0.0187	0.38	2
L_349_2101	349	gurobi:dual-simplex	0.017	0.0007	0.03	20
L_349_2101	349	gurobi:barrier	0.028	0.0215	0.44	2
L_349_2101	349	cplex-cli:dual-simplex	0.025	0.0003	0.03	20
M_538_3033	538	path	0.035	0.0000	0.03	
M_538_3033	538	newton	0.033	0.0046	0.12	2
M_538_3033	538	highs:dual-simplex	0.037	0.0010	0.06	1
M_538_3033	538	highs:ipm	0.041	0.0314	0.64	1
M_538_3033	538	gurobi:dual-simplex	0.031	0.0010	0.05	5
M_538_3033	538	gurobi:barrier	0.056	0.0469	0.95	1
M_538_3033	538	cplex-cli:dual-simplex	0.059	0.0004	0.07	1
A_972_1661	972	path	0.056	0.0000	0.06	
A_972_1661	972	newton	0.029	0.0041	0.11	8
A_972_1661	972	highs:dual-simplex	0.012	0.0013	0.04	37
A_972_1661	972	highs:ipm	0.027	0.0218	0.44	3
A_972_1661	972	gurobi:dual-simplex	0.011	0.0010	0.03	49
A_972_1661	972	gurobi:barrier	0.021	0.0150	0.31	4
A_972_1661	972	cplex-cli:dual-simplex	0.018	0.0007	0.03	54
B_981_1688	981	path	0.051	0.0000	0.05	
B_981_1688	981	newton	0.032	0.0043	0.11	6
B_981_1688	981	highs:dual-simplex	0.013	0.0015	0.04	26
B_981_1688	981	highs:ipm	0.030	0.0226	0.46	2
B_981_1688	981	gurobi:dual-simplex	0.013	0.0013	0.04	32
B_981_1688	981	gurobi:barrier	0.020	0.0154	0.31	4
B_981_1688	981	cplex-cli:dual-simplex	0.016	0.0009	0.03	41
newman1	1060	path	0.044	0.0000	0.04	
newman1	1060	newton	0.034	0.0132	0.15	2
newman1	1060	highs:dual-simplex	0.067	0.0006	0.07	1
newman1	1060	highs:ipm	0.090	0.0558	0.59	1
newman1	1060	gurobi:dual-simplex	0.050	0.0011	0.06	1
newman1	1060	gurobi:barrier	0.054	0.0440	0.45	1
newman1	1060	cplex-cli:dual-simplex	0.111	0.0003	0.11	1
N_1217_7616	1217	path	0.058	0.0000	0.06	
N_1217_7616	1217	newton	0.039	0.0098	0.23	3
N_1217_7616	1217	highs:dual-simplex	0.059	0.0056	0.17	1
N_1217_7616	1217	highs:ipm	0.084	0.0743	1.50	1
N_1217_7616	1217	gurobi:dual-simplex	0.067	0.0038	0.14	1
N_1217_7616	1217	gurobi:barrier	0.076	0.0522	1.07	1
N_1217_7616	1217	cplex-cli:dual-simplex	0.064	0.0032	0.12	1
C_1336_2382	1336	path	0.087	0.0000	0.09	
C_1336_2382	1336	newton	0.039	0.0075	0.18	8
C_1336_2382	1336	highs:dual-simplex	0.021	0.0026	0.07	28
C_1336_2382	1336	highs:ipm	0.055	0.0454	0.92	2
C_1336_2382	1336	gurobi:dual-simplex	0.023	0.0018	0.06	38
C_1336_2382	1336	gurobi:barrier	0.037	0.0281	0.57	3
C_1336_2382	1336	cplex-cli:dual-simplex	0.036	0.0016	0.07	34
O_1711_11661	1711	path	0.043	0.0000	0.04	
O_1711_11661	1711	newton	0.045	0.0154	0.34	1
O_1711_11661	1711	highs:dual-simplex	0.284	0.0067	0.41	1
O_1711_11661	1711	highs:ipm	0.237	0.1955	3.95	1
O_1711_11661	1711	gurobi:dual-simplex	0.255	0.0047	0.34	1
O_1711_11661	1711	gurobi:barrier	0.261	0.1514	3.14	1
O_1711_11661	1711	cplex-cli:dual-simplex	0.094	0.0032	0.16	1
D_1790_3130	1790	path	0.074	0.0000	0.07	

instance	n	method	first (s)	per extra (s)	20 caps (s)	break-even
D_1790_3130	1790	newton	0.033	0.0065	0.16	8
D_1790_3130	1790	highs:dual-simplex	0.030	0.0038	0.10	13
D_1790_3130	1790	highs:ipm	0.077	0.0512	1.05	1
D_1790_3130	1790	gurobi:dual-simplex	0.025	0.0022	0.07	24
D_1790_3130	1790	gurobi:barrier	0.043	0.0341	0.69	2
D_1790_3130	1790	cplex-cli:dual-simplex	0.037	0.0025	0.08	16
E_1790_3130	1790	path	0.069	0.0000	0.07	
E_1790_3130	1790	newton	0.032	0.0064	0.15	7
E_1790_3130	1790	highs:dual-simplex	0.029	0.0034	0.09	13
E_1790_3130	1790	highs:ipm	0.075	0.0495	1.02	1
E_1790_3130	1790	gurobi:dual-simplex	0.028	0.0024	0.07	18
E_1790_3130	1790	gurobi:barrier	0.046	0.0309	0.63	2
E_1790_3130	1790	cplex-cli:dual-simplex	0.037	0.0025	0.08	14
F_3091_5715	3091	path	0.183	0.0000	0.18	
F_3091_5715	3091	newton	0.048	0.0097	0.23	16
F_3091_5715	3091	highs:dual-simplex	0.058	0.0060	0.17	22
F_3091_5715	3091	highs:ipm	0.137	0.0738	1.54	2
F_3091_5715	3091	gurobi:dual-simplex	0.038	0.0040	0.11	38
F_3091_5715	3091	gurobi:barrier	0.072	0.0588	1.19	3
F_3091_5715	3091	cplex-cli:dual-simplex	0.075	0.0059	0.19	20
G_3091_5715	3091	path	0.079	0.0000	0.08	
G_3091_5715	3091	newton	0.034	0.0158	0.33	4
G_3091_5715	3091	highs:dual-simplex	0.056	0.0100	0.25	4
G_3091_5715	3091	highs:ipm	0.143	0.1063	2.16	1
G_3091_5715	3091	gurobi:dual-simplex	0.048	0.0076	0.19	6
G_3091_5715	3091	gurobi:barrier	0.097	0.0651	1.33	1
G_3091_5715	3091	cplex-cli:dual-simplex	0.076	0.0078	0.22	2
H_3091_5715	3091	path	0.258	0.0000	0.26	
H_3091_5715	3091	newton	0.040	0.0123	0.27	19
H_3091_5715	3091	highs:dual-simplex	0.035	0.0087	0.20	27
H_3091_5715	3091	highs:ipm	0.177	0.0852	1.80	2
H_3091_5715	3091	gurobi:dual-simplex	0.032	0.0047	0.12	49
H_3091_5715	3091	gurobi:barrier	0.091	0.0609	1.25	4
H_3091_5715	3091	cplex-cli:dual-simplex	0.037	0.0061	0.15	38
I_3091_5715	3091	path	0.092	0.0000	0.09	
I_3091_5715	3091	newton	0.033	0.0107	0.24	7
I_3091_5715	3091	highs:dual-simplex	0.055	0.0115	0.27	5
I_3091_5715	3091	highs:ipm	0.159	0.1344	2.71	1
I_3091_5715	3091	gurobi:dual-simplex	0.038	0.0064	0.16	10
I_3091_5715	3091	gurobi:barrier	0.088	0.0618	1.26	2
I_3091_5715	3091	cplex-cli:dual-simplex	0.061	0.0086	0.22	5
P_3243_22306	3243	path	0.071	0.0000	0.07	
P_3243_22306	3243	newton	0.063	0.0362	0.75	2
P_3243_22306	3243	highs:dual-simplex	0.960	0.0280	1.49	1
P_3243_22306	3243	highs:ipm	0.627	0.4104	8.43	1
P_3243_22306	3243	gurobi:dual-simplex	0.445	0.0110	0.65	1
P_3243_22306	3243	gurobi:barrier	0.336	0.2892	5.83	1
P_3243_22306	3243	cplex-cli:dual-simplex	0.345	0.0121	0.57	1
Q_3428_19555	3428	path	0.176	0.0000	0.18	
Q_3428_19555	3428	newton	0.057	0.0266	0.56	6
Q_3428_19555	3428	highs:dual-simplex	0.304	0.0386	1.04	1
Q_3428_19555	3428	highs:ipm	0.280	0.2339	4.72	1
Q_3428_19555	3428	gurobi:dual-simplex	0.291	0.0218	0.71	1
Q_3428_19555	3428	gurobi:barrier	0.321	0.2117	4.34	1
Q_3428_19555	3428	cplex-cli:dual-simplex	0.432	0.0315	1.03	1
R_4281_24452	4281	path	0.223	0.0000	0.22	
R_4281_24452	4281	newton	0.058	0.0317	0.66	7
R_4281_24452	4281	highs:dual-simplex	0.445	0.0562	1.51	1
R_4281_24452	4281	highs:ipm	0.383	0.3394	6.83	1
R_4281_24452	4281	gurobi:dual-simplex	0.439	0.0362	1.13	1
R_4281_24452	4281	gurobi:barrier	0.316	0.2687	5.42	1

instance	n	method	first (s)	per extra (s)	20 caps (s)	break-even
R_4281_24452	4281	cplex-cli:dual-simplex	0.545	0.0401	1.31	1
S_5624_36504	5624	path	0.231	0.0000	0.23	
S_5624_36504	5624	newton	0.073	0.0452	0.93	5
S_5624_36504	5624	highs:dual-simplex	0.557	0.0766	2.01	1
S_5624_36504	5624	highs:ipm	0.644	0.6134	12.30	1
S_5624_36504	5624	gurobi:dual-simplex	0.628	0.0539	1.65	1
S_5624_36504	5624	gurobi:barrier	0.437	0.3956	7.95	1
S_5624_36504	5624	cplex-cli:dual-simplex	0.780	0.0605	1.93	1
T_6271_42080	6271	path	0.249	0.0000	0.25	
T_6271_42080	6271	newton	0.102	0.0633	1.31	4
T_6271_42080	6271	highs:dual-simplex	1.851	0.1389	4.49	1
T_6271_42080	6271	highs:ipm	0.994	0.9243	18.56	1
T_6271_42080	6271	gurobi:dual-simplex	1.281	0.0911	3.01	1
T_6271_42080	6271	gurobi:barrier	0.792	0.7597	15.23	1
T_6271_42080	6271	cplex-cli:dual-simplex	1.426	0.1233	3.77	1
U_6494_48626	6494	path	0.160	0.0000	0.16	
U_6494_48626	6494	newton	0.101	0.0605	1.25	2
U_6494_48626	6494	highs:dual-simplex	4.511	0.1008	6.43	1
U_6494_48626	6494	highs:ipm	1.927	1.8291	36.68	1
U_6494_48626	6494	gurobi:dual-simplex	3.420	0.0301	3.99	1
U_6494_48626	6494	gurobi:barrier	1.373	1.0841	21.97	1
U_6494_48626	6494	cplex-cli:dual-simplex	1.332	0.0243	1.79	1
J_9235_17082	9235	path	0.336	0.0001	0.34	
J_9235_17082	9235	newton	0.062	0.0308	0.65	10
J_9235_17082	9235	highs:dual-simplex	0.194	0.0836	1.78	3
J_9235_17082	9235	highs:ipm	1.027	0.5370	11.23	1
J_9235_17082	9235	gurobi:dual-simplex	0.219	0.0485	1.14	4
J_9235_17082	9235	gurobi:barrier	0.615	0.5192	10.48	1
J_9235_17082	9235	cplex-cli:dual-simplex	0.217	0.0667	1.48	3
K_9235_17082	9235	path	0.404	0.0001	0.41	
K_9235_17082	9235	newton	0.069	0.0360	0.75	11
K_9235_17082	9235	highs:dual-simplex	0.260	0.0721	1.63	3
K_9235_17082	9235	highs:ipm	1.015	0.5246	10.98	1
K_9235_17082	9235	gurobi:dual-simplex	0.223	0.0361	0.91	7
K_9235_17082	9235	gurobi:barrier	0.537	0.4861	9.77	1
K_9235_17082	9235	cplex-cli:dual-simplex	0.367	0.0641	1.59	2
zuck_small	9400	path	0.448	0.0000	0.45	
zuck_small	9400	newton	0.264	0.2121	2.17	2
zuck_small	9400	highs:dual-simplex	10.856	2.2758	31.34	1
zuck_small	9400	highs:ipm	3.830	3.3173	33.69	1
zuck_small	9400	gurobi:dual-simplex	13.848	0.6427	19.63	1
zuck_small	9400	gurobi:barrier	15.041	17.1200	169.12	1
zuck_small	9400	cplex-cli:dual-simplex	5.557	1.9086	22.73	1
V_10001_63944	10001	path	0.470	0.0000	0.47	
V_10001_63944	10001	newton	0.152	0.1085	2.21	4
V_10001_63944	10001	highs:dual-simplex	3.004	0.3430	9.52	1
V_10001_63944	10001	highs:ipm	2.360	1.5793	32.37	1
V_10001_63944	10001	gurobi:dual-simplex	2.042	0.1871	5.60	1
V_10001_63944	10001	gurobi:barrier	1.091	0.9488	19.12	1
V_10001_63944	10001	cplex-cli:dual-simplex	2.883	0.2725	8.06	1
W_11757_83218	11757	path	0.420	0.0000	0.42	
W_11757_83218	11757	newton	0.191	0.1693	3.41	3
W_11757_83218	11757	highs:dual-simplex	7.296	0.5810	18.34	1
W_11757_83218	11757	highs:ipm	3.586	3.3430	67.10	1
W_11757_83218	11757	gurobi:dual-simplex	5.958	0.4925	15.32	1
W_11757_83218	11757	gurobi:barrier	2.856	2.0426	41.66	1
W_11757_83218	11757	cplex-cli:dual-simplex	5.156	0.3702	12.19	1
kd	14153	path	1.162	0.0001	1.16	
kd	14153	newton	0.258	0.2150	2.19	6
kd	14153	highs:dual-simplex	10.158	4.8421	53.74	1
kd	14153	highs:ipm	6.682	5.3154	54.52	1

instance	n	method	first (s)	per extra (s)	20 caps (s)	break-even
kd	14153	gurobi:dual-simplex	22.811	2.1317	42.00	1
kd	14153	gurobi:barrier	11.764	8.8330	91.26	1
kd	14153	cplex-cli:dual-simplex	13.911	7.8421	84.49	1
zuck_medium	29277	path	3.155	0.0001	3.16	
zuck_medium	29277	newton	2.243	1.8489	18.88	2
zuck_medium	29277	highs:dual-simplex	-	-	420.00	
zuck_medium	29277	highs:ipm	-	-	420.00	
zuck_medium	29277	gurobi:dual-simplex	208.501	10.1115	299.51	1
zuck_medium	29277	gurobi:barrier	-	-	420.00	
zuck_medium	29277	cplex-cli:dual-simplex	-	-	420.00	
p4hd	40947	path	11.361	0.0002	11.36	
p4hd	40947	newton	0.806	0.6895	7.01	17
p4hd	40947	highs:dual-simplex	-	-	150.00	
p4hd	40947	highs:ipm	-	-	150.00	
p4hd	40947	gurobi:dual-simplex	-	-	150.00	
p4hd	40947	gurobi:barrier	-	-	150.00	
p4hd	40947	cplex-cli:dual-simplex	-	-	150.00	
marvin	53271	path	3.335	0.0001	3.34	
marvin	53271	newton	0.840	0.4892	5.24	7
marvin	53271	highs:dual-simplex	10.822	3.9953	46.78	1
marvin	53271	highs:ipm	4.855	4.3104	43.65	1
marvin	53271	gurobi:dual-simplex	17.237	0.9451	25.74	1
marvin	53271	gurobi:barrier	12.941	11.5829	117.19	1
marvin	53271	cplex-cli:dual-simplex	5.504	2.7759	30.49	1
w23	74260	path	12.405	0.0003	12.41	
w23	74260	newton	0.936	1.5492	14.88	9
w23	74260	highs:dual-simplex	-	-	150.00	
w23	74260	highs:ipm	-	-	150.00	
w23	74260	gurobi:dual-simplex	-	-	150.00	
w23	74260	gurobi:barrier	-	-	150.00	
w23	74260	cplex-cli:dual-simplex	-	-	150.00	
zuck_large	96821	path	24.998	0.0004	25.00	
zuck_large	96821	newton	2.983	3.1034	30.91	9
zuck_large	96821	highs:dual-simplex	-	-	150.00	
zuck_large	96821	highs:ipm	-	-	150.00	
zuck_large	96821	gurobi:dual-simplex	-	-	150.00	
zuck_large	96821	gurobi:barrier	-	-	150.00	
zuck_large	96821	cplex-cli:dual-simplex	-	-	150.00	
sm2	99014	path	2.375	0.0002	2.38	
sm2	99014	newton	0.314	0.1265	1.45	18
sm2	99014	highs:dual-simplex	0.812	0.2165	2.76	9
sm2	99014	highs:ipm	1.151	1.1131	11.17	3
sm2	99014	gurobi:dual-simplex	0.359	0.0962	1.22	22
sm2	99014	gurobi:barrier	0.356	0.2010	2.17	12
sm2	99014	cplex-cli:dual-simplex	1.671	0.4838	6.02	3
mclaughlin_limit	112687	path	46.497	0.0003	46.50	
mclaughlin_limit	112687	newton	3.695	4.0239	39.91	12
mclaughlin_limit	112687	highs:dual-simplex	-	-	150.00	
mclaughlin_limit	112687	highs:ipm	-	-	150.00	
mclaughlin_limit	112687	gurobi:dual-simplex	-	-	150.00	
mclaughlin_limit	112687	gurobi:barrier	-	-	150.00	
mclaughlin_limit	112687	cplex-cli:dual-simplex	-	-	150.00	

Table 5: What to use, and on what evidence

question	answer	
one capacity	Newton search on the weight price	fastest on 23/33 instances;
the whole value function	the canonical path	fastest on 24/33 instances; median 2.4x the f
an LP solver is needed	dual simplex, never barrier	barrier cannot warm start, so its margi